

HiDiTingV100 资源部署

用户指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档详细描述了 HiDiTingV100 的资源部署方案，包括镜像、内存等，并给出内存分析工具及调优方法，供用户参考进行产品适配。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 镜像分区表.....1

2 内存部署.....3

3 内存分析工具7

3.1 静态内存..... 7

3.2 动态内存..... 8

4 常见内存优化方法9

插图目录

图 2-1 代码段部署至 SRAM..... 4

图 2-2 psram_data 段部署 5

图 2-3 psram_bss 段部署 5

图 2-4 任务栈支持 DTCM/PSRAM 部署 6

图 3-1 根据静态库粒度进行内存统计修改示意 8

表格目录

表 1-1 镜像分区表..... 1

表 3-1 堆栈内存信息查看命令..... 8

表 4-1 SDK 可裁剪组件 9

1 镜像分区表

当前镜像默认存储在外部 NorFlash 上，并按照 8M NorFlash 容量进行镜像地址划分。
EVB 单板上镜像地址信息请参考文件
“drivers\boards\brandy_evb\memory_config\include\memory_config_common.h”。镜
像分区表如表 1-1 所示。

表1-1 镜像分区表

名称	存储设备	地址	长度
ssb	norflash	0x00000000	0x00020000
ssb_backup	norflash	0x00020000	0x00020000
security_signature	norflash	0x00040000	0x0000F000
partition_table	norflash	0x0004F000	0x00001000
recovery	norflash	0x00050000	0x00040000
bt	norflash	0x00090000	0x00070000
dsp	norflash	0x00100000	0x00070000
dsp_overlay	norflash	0x00170000	0x00140000
application	norflash	0x002B0000	0x00549000
nv	norflash	0x007F9000	0x00004000
ota_magic_code	norflash	0x007FD000	0x00003000

须知

- 由于 BT、DSP 镜像对外闭源交付，起始地址固定，因此除 application 镜像外，不建议用户自行修改其余地址划分，需要修改时请告知 FAE。
- 分区表无法通过 OTA 升级更改，因此要注意，分区表的划分需要在产品出货前确定。

2 内存部署

芯片可使用内存可具体分为 ITCM/DTCM、L2MEM、PSRAM 三部分，其中 ITCM/DTCM MCU 独占，L2MEM 与 DSP、GPU 共享。各类内存属性如下，访问性能 ITCM/DTCM > L2MEM > PSRAM，读写功耗 ITCM/DTCM < L2MEM < PSRAM，大小 ITCM/DTCM < L2MEM < PSRAM，实际综合性能、功耗、大小进行内存部署。

内存部署具体信息可参考链接脚本

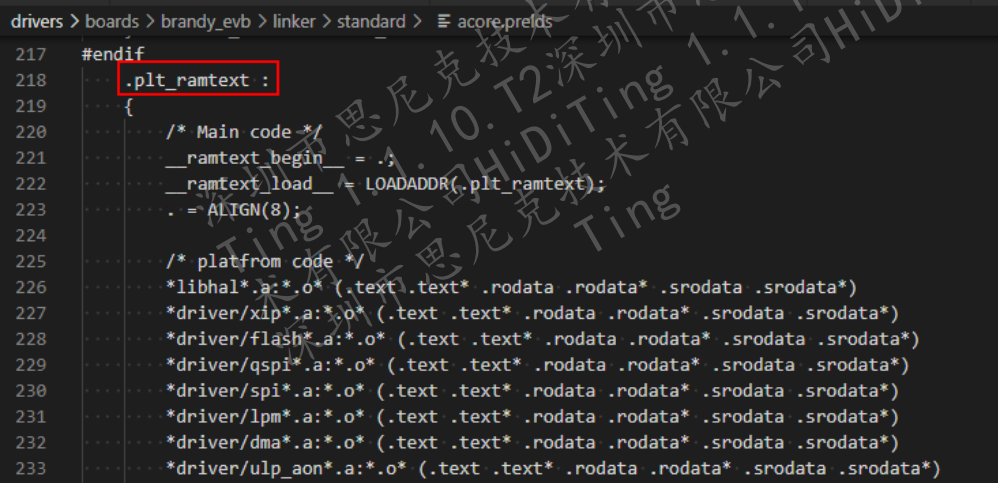
“drivers\boards\brandy_evb\linker\standard\acore.prellds”。

- 代码段部署

支持 Flash XIP 特性，因此绝大部分代码直接运行在 XIP 上，只有少量代码需要重定向到芯片内部 SRAM 里边运行，包括 Flash 驱动代码、低功耗相关代码以及部分基础库。

如需将代码部署到芯片 SRAM 上，即在 “.plt_ramtext” 段中增加对应静态库即可，具体可参考图 2-1。

图2-1 代码段部署至 SRAM



```
drivers > boards > brandy_evb > linker > standard > acore.prelds
217 #endif
218 .plt_ramtext :
219 {
220     /* Main code */
221     _ramtext_begin = .;
222     _ramtext_load = LOADADDR(.plt_ramtext);
223     . = ALIGN(8);
224
225     /* platform code */
226     *libhal*.a:*.*.o* (.text .text* .rodata .rodata* .srodata .srodata*)
227     *driver/xip*.a:*.*.o* (.text .text* .rodata .rodata* .srodata .srodata*)
228     *driver/flash*.a:*.*.o* (.text .text* .rodata .rodata* .srodata .srodata*)
229     *driver/qspi*.a:*.*.o* (.text .text* .rodata .rodata* .srodata .srodata*)
230     *driver/spi*.a:*.*.o* (.text .text* .rodata .rodata* .srodata .srodata*)
231     *driver/lpm*.a:*.*.o* (.text .text* .rodata .rodata* .srodata .srodata*)
232     *driver/dma*.a:*.*.o* (.text .text* .rodata .rodata* .srodata .srodata*)
233     *driver/ulp_aon*.a:*.*.o* (.text .text* .rodata .rodata* .srodata .srodata*)
```

- 静态数据段部署

静态数据段包括 data 段和 bss 段。

- bss 段指初始化为 0 或者未初始化的全局变量。
- data 段指初始化非 0 的全局变量。

两者均默认部署至内部 SRAM 上。当芯片内部 SRAM 空间不足时，也可将数据段部署至外部 PSRAM 上，具体可参考图 2-2、图 2-3。

📖 说明

外部 PSRAM 内存更大，但是访问性能低于 SRAM，功耗也高于 SRAM，因此考虑系统低功耗，适合将访问频次低或亮屏执行的部分放至 PSRAM 上。

图2-2 psram_data 段部署

```

drivers > boards > brandy_evb > linker > standard > acore.prelds
81 < #ifdef CONFIG_PSRAM_SUPPORT
82 <     .psram_data :
83 <     {
84 <         __psram_data_start = .;
85 <         __psram_data_load = LOADADDR(.psram_data);
86 <         . = ALIGN(8);
87 <         SORT(*libjerryscript*.a:*.*.*) (.data .data* .sdata .sdata*)
88 <         SORT(*libpermission*.a:*.*.*) (.data .data* .sdata .sdata*)
89 <         SORT(*libappverify*.a:*.*.*) (.data .data* .sdata .sdata*)
90 <         SORT(*libbootstrap*.a:*.*.*) (.data .data* .sdata .sdata*)
91 <         SORT(*libsamgr*.a:*.*.*) (.data .data* .sdata .sdata*)
92 <         SORT(*libabms*.a:*.*.*) (.data .data* .sdata .sdata*)
93 <         SORT(*libace*.a:*.*.*) (.data .data* .sdata .sdata*)
94 <         SORT(*libresmgr*.a:*.*.*) (.data .data* .sdata .sdata*)
95 <         SORT(*libi18n*.a:*.*.*) (.data .data* .sdata .sdata*)
96 <         SORT(*libsyspara*.a:*.*.*) (.data .data* .sdata .sdata*)
97 <         SORT(*application/wearable*.a:*.*.*) (.data .data*)
98 <         SORT(*application/wstp*.a:*.*.*) (.data .data* .sdata .sdata*)
99 <         . = ALIGN(8);
100 <         __psram_data_end = .;
101 <     } > PSRAM_APP_DATA AT > FLASH_PROGRAM
102 <     __psram_data_size = __psram_data_end - __psram_data_start;

```

图2-3 psram_bss 段部署

```

.psram_bss (NOLOAD):
{
    __psram_bss_start = .;
    __psram_bss_load = LOADADDR(.psram_bss);
    . = ALIGN(8);
    SORT(*libjerryscript*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    SORT(*libpermission*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    SORT(*libappverify*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    SORT(*libbootstrap*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    SORT(*libsamgr*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    SORT(*libabms*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    SORT(*libace*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    SORT(*libresmgr*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    SORT(*libi18n*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    SORT(*libsyspara*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    SORT(*application/wearable*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    SORT(*application/wstp*.a:*.*.*) (.bss .bss* .sbss .sbss* COMMON)
    . = ALIGN(8);
    __psram_bss_end = .;
} > PSRAM_APP_DATA
__psram_bss_size = __psram_bss_end - __psram_bss_start;

```

- 栈空间

系统中断栈、异常栈默认部署在 DTCM，不支持修改；任务栈支持部署在 DTCM 或者 PSRAM 上，具体示例如图 2-4 所示。

图2-4 任务栈支持 DTCM/PSRAM 部署

```
app_task_definition_t g_app_tasks[] = {
    { { "app", 0, NULL, 0, NULL, APP_MAIN_STACK_SIZE, TASK_PRIORITY_APP, 0, 0 },
      (osThreadFunc_t)app_main, NULL, NULL, USE_DTCM_MEM },
    #if (USE_COMPRESS_LOG_INSTEAD_OF_SDT_LOG == NO)
    { { "log", 0, NULL, 0, NULL, (0xC00), TASK_PRIORITY_LOG, 0, 0 },
      (osThreadFunc_t)log_main, NULL, NULL, USE_DTCM_MEM },
    #endif
    { { "bt_sdk", 0, NULL, 0, NULL, BTH_SDK_STACK_SIZE, TASK_PRIORITY_SDK, 0, 0 },
      (osThreadFunc_t)bt_acore_task_main, NULL, NULL, USE_DTCM_MEM },
    #ifdef SUPPORT_AT_CMD
    { { "at_cmd", 0, NULL, 0, NULL, 0xC00, TASK_PRIORITY_APP, 0, 0 },
      (osThreadFunc_t)at_cmd_task_process_handler, NULL,
      (CreateQueueFunction_t)at_cmd_task_queue_init, USE_EXTEND_MEM },
    #endif
}
```

- 堆空间

堆空间分为两部分，一部分为 kernel OS 内部使用的系统堆空间，默认部署至内部 SRAM 上；另一部分为各个任务使用的内存堆空间，如各个任务 malloc 申请空间，默认部署至外部 PSRAM 上，具体可参考

“drivers\boards\brandy_evb\memory_config\include\psram.h” 头文件。

此外，堆空间支持自定义内存池使用，调用 osal 接口 osal_pool_mem_init、osal_pool_mem_alloc，或者 cmsis 接口 osMemoryPoolNew、osMemoryPoolAlloc 实现。

3 内存分析工具

3.1 静态内存

3.2 动态内存

3.1 静态内存

IDE 工具提供了镜像分析功能，可以方便分析数据段、代码段等静态内存占用情况。

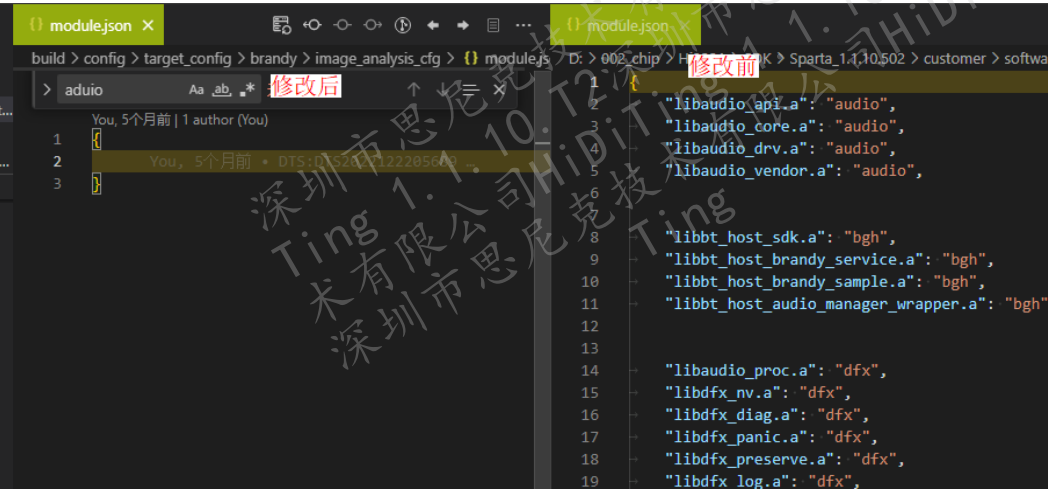
具体使用方法请参见《HiDiTingV100 IDE 工具 使用指南》“镜像分析”章节。

其中镜像分析功能中默认按照模块进行数据统计，如 graphic、media、driver 等模块。模块划分列表可参考

“build\config\target_config\brandy\image_analysis_cfg\module.json”文件。

如需要按照静态库 “.a” 文件的粒度进行内存统计，则可以将 “module.json” 文件中对应分组信息删除，修改如[图 3-1](#)所示。

图3-1 根据静态库粒度进行内存统计修改示意



3.2 动态内存

支持 AT 命令查看堆、栈内存信息，具体命令如下：

表3-1 堆栈内存信息查看命令

AT 命令	描述
AT^HEAPINFO	系统堆内存信息查看。
AT^STACKINFO	系统任务栈内存信息查看，包括任务栈名称、大小、峰值、使用情况等。

4 常见内存优化方法

- SDK 组件裁剪
SDK 当前除功能代码外，默认包含大量测试代码、维测代码以及 Sample 示例代码，实际用户集成 SDK 时，不需要该部分功能，因此可直接裁剪。具体可裁剪组件如表 4-1 所示。

表4-1 SDK 可裁剪组件

组件名	对应功能宏	功能描述
'graphic_at_service'	-	图形测试组件。
'app_at_service'	-	系统 BSP 测试组件。
'audio_at_service'	-	音频测试组件。
'media_at_service'	-	媒体测试组件。
'bt_manager_at_service'	-	蓝牙测试组件。
'audio_proc'	'CONFIG_ZDIAG_AUDIO_PROC_SUPPORT'	音频维测相关组件。
'audio_dump'	'CONFIG_ZDIAG_AUDIO_DUMP_SUPPORT'	音频维测相关组件。
'audio_probe'	'CONFIG_ZDIAG_AUDIO_PROBE_SUPPORT'	音频维测相关组件。
	'SUPPORT_GPU_OPEN_VG'	GPU 支持 OpenVG 接口。

组件名	对应功能宏	功能描述
	'SW_UART_DEBUG'	支持串口直接打印。

- 内部部署调整

内部 SRAM 空间不足时，可以将部分数据段、任务栈部署至外部 PSRAM。

- 代码优化

结合内存分析工具，排查当前哪一个模块占用内存较大，结合代码进行优化。代码优化手段较多，此处列出常见的几种优化手段帮助用户优化内存。

- 局部变量不应过大，会撑大栈空间，可考虑使用动态 malloc 的形式申请内存。
- 超大全局变量数组，需要考虑合理性，可考虑修改为链表的形式进行动态申请。
- 根据栈峰值使用情况，合理规划栈大小，根据经验，中等复杂程度任务一般栈空间 3KB 左右即可。
- 正常的 printf 接口无法对打印字符串进行压缩，会导致 CodeSize 迅速膨胀，因此可使用 Diag 压缩日志接口打印。
- 将不变的数据段尽量修改为 const 类型，即由 data 段转变为 rodata 段，内存部署也从内部 SRAM 转变为 NorFlash。